Guidelines for Submission to MICRO 2026

MICRO 2026 Submission **#NaN** – Confidential Draft – Do NOT Distribute!!

Abstract

Keywords

Do, Not, Use, This, Code, Put, the, Correct, Terms, for, Your, Paper

1 Introduction

Memory-level parallelism (MLP) is important for efficient out-of-order (OoO) execution and overcoming the memory wall in processor performance. Memory dependence prediction is a speculative technique used in OoO processors that aims to enhance total MLP by predicting the stores on which a given load instruction will depend, as well as identifying memory-independent loads. Smaller cores utilising smaller predictors can often struggle with a large number of false dependencies, which unnecessarily stalls loads on non-dependent stores. This is often due to hash collisions in small predictor tables, or simple predictor algorithms that cannot capture varying dependency behaviour. The Store Sets predictor [**?**] is an example with both of these issues, and variations of the algorithm is found in real processors today [**?**]. Whereas modern predictors like PHAST [**?**] have pushed the SotA to address these issues via techniques like associative tagging and context based predictions, they place higher area, energy and complexity demands on core design, which is often not viable in smaller cores. At the same time, while simply growing a Store Sets-like predictor would offset some of the hash collisions, this benefit struggles to outperform an iso-area comparison to scaling other components. As such, memory dependence predictors in small cores typically remain small and inefficient.

The role of such area-constrained cores has grown over time. Modern processors increasingly deploy heterogeneous and efficiency-oriented core designs, combining larger high-performance cores with small energy-efficient (but still out-of-order) cores, as seen in ARM big.LITTLE systems and both Apple's & Intel's performance and efficiency cores. Efficiency-focused cores are, of course, also found in power-constrained systems such as smartphones, tablets, smartwatches and other edge systems. These cores are more and more performing demanding general purpose computation while striving to consume as little energy as possible. As they grow more performance-sensitive, finding optimisations which don't also incur additional area, power or complexity costs becomes more pressing.

Prior work demonstrates that a considerable portion of load instructions in general-purpose workloads rarely or never exhibit in-flight memory dependencies [**? ? ?**]. This paper uses this observation to reduce false dependencies in memory dependence prediction via a profile-guided software co-design at zero hardware cost. Our technique labels frequently memory-independent loads by their opcode, which then bypass MDP queries and are always scheduled as early as possible. We show that the vast majority of dynamic loads can avoid querying the MDP while maintaining or even improving IPC, as well as providing a potential power saving.

MICRO 2026, Athens, Greece
2026. ACM ISBN 978-X-XXXX-XXXX-X/XX/XX
https://doi.org/XXXXXXX.XXXXXXX

1.1 Contributions

This paper makes the following contributions:

- Implements a PGO technique to label memory independent loads to the OoO core at no-cost communication or hardware overhead.
- Implements a modified Gem5 to simulate labelled loads against different MDP algorithms and core size configurations, as well as extends McPat to provide power usage estimations of the MDP unit.
- Evaluates Spec2017 IntSpeed and demonstrates on average: 72% of MDP load queries are redundant, IPC improves by 0.47% & MDP power reduces by 7% on the smallest core, whereas the largest core maintains IPC and reduces MDP power by 42%.
  - Additionally demonstrates a 3.2% IPC gain on highly constrained cores that lack any memory dependence prediction.
  - Additionally demonstrates a x% IPC gain when modelling a limited number of MDP read ports.
- (currently only includes 2 benchmarks) Evaluates Spec2017 FloatSpeed and demonstrates on average: 87% of MDP load queries are redundant, IPC improves by 0.1% & MDP power reduces by 6.5% on the smallest core, whereas the largest core maintains IPC and reduces MDP power by x%.
  - Additionally demonstrates a 16.5% IPC gain on highly constrained cores that lack any memory dependence prediction.
  - Additionally demonstrates a x% IPC gain when modelling a limited number of MDP read ports.

2 Background & Related Work

- LSQ, MDP, store sets, PHAST
- my original ARCS paper
- store distances paper & comparison to us
- constable
- LLVM store queue search skip paper
- LSQ scalability labelling paper
- software-hardware cooperative memory disamgibuation (much more overhead for much less gain, doesn't address multi-cores, only evaluates floatspeed)

3 Method

- justifying PGO: how PGO is commonly used and static analysis (even SVF) doesn't work
- where we win (read only/likely stable, hash collisions, resolved stores, occasional dependence)

4 Evaluation

- experimental design
– cpu size selection reasoning
- IPC
- False deps
- Lookups
- Energy
- Violations
- Read ports - PGO & threshold distance scaling
- Prior store distance work comparison
- Constable coverage comparison

This section presents the experimental results using our PGO technique. It highlights the extent of redundant memory dependence predictor queries found in general-purpose workloads and their impact on IPC and power usage. Our results demonstrate that a large portion of predictor activity can be avoided, leading to improvements in performance, power usage, and even that PND load labels can serve as a partial replacement for the memory dependence predictor unit, all with zero hardware overhead. Furthermore, we examine the generalisability of our generated store distance profiles to unseen inputs and provide a comparison to similar prior work [?].

4.1 Experimental Design

We use Gem5 to simulate varying core size configurations to measure impact as components scale, going from a very small core (ROB=64) to a large core (ROB=512). Each core either utilises the XS Store Sets or PHAST predictor, depending on which provides better performance for a realistic area budget at that size. We find that XS Store Sets performs optimally in smaller area allocations, whereas PHAST is more accurate when allowed larger areas. Additionally, we model the v. small configuration to not use a memory dependence predictor at all, and instead conservatively stalls loads on all unresolved stores. This shows the impact on highly constrained cores that cannot spare the overhead of memory dependence predictor whatsoever.

The full configuration for each model is detailed in Table 1. All models except the v. small core utilise a 64KB TAGE-SC-L for branch prediction and a DCPT prefetcher. To more accurately model the constrained size of the v. small core, it employs a tournament predictor with 2k local and 8k global entries and a stride prefetcher. We evaluate the Spec2017 Intspeed and Floatspeed benchmarks using up to 10 simpoints, with an interval of 100M instructions and a warm-up of 10M. Gem5 is modified to bypass MDP lookups for labelled loads and to avoid creating new entries in the case of a memory order violation. Violating labelled loads still roll back as normal and do not alter program semantics. The Gem5 results are subsequently processed by an extended McPat, which models Store Sets or PHAST as applicable (or upstream McPat for the v. small core).

4.2 IPC

4.3 MDP Queries

Average of loads dispatched and hence accessing the predicor per cycle. Mention consequences: Number of ports used, and hence energy; limit in the number of loads per cycle being able to be dispatched.

Table 1: Parameters of processor components for each simulated model

		V. Small	Small	Med.	Large
Instruction Window	ROB: IQ: LQ: SQ:	64 52 21 12	128 77 38 22	256 154 77 54	512 308 154 108
Pipeline Width	Fetch/ Commit: Issue:	3 6	4 8	6 8	8 12
L1 Cache	L1D (KB): L1I (KB): MSHRs:	16 16 4	32 32 8	64 32 16	64 64 32
L2 Cache	L2 (MB): MSHRs:	1 8	2 16	4 32	8 64
XS Store Sets	SSIT: LFST: Slots: Clear:	N/A	64 32 2 31232	128 32 4 31232	N/A
PHAST	Rows: Assoc: Tag Bits: Counter Bits:	N/A	N/A	N/A	32 2 8 2
MDP Read Ports:		N/A	1	2	4

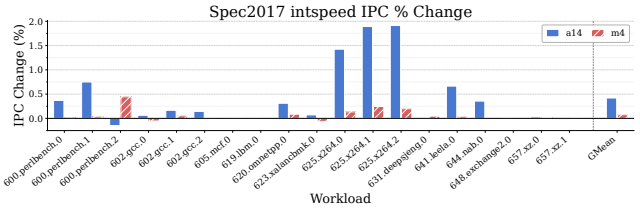

Figure 1: IPC % changes for each CPU model on each intspeed benchmark.

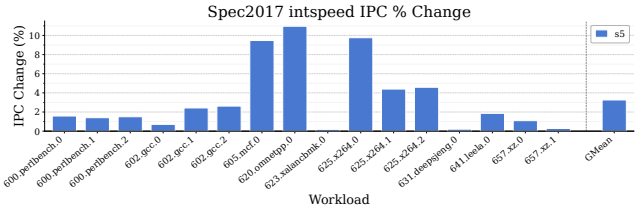

Figure 2: IPC % changes for the no MDP S5 model on each intspeed benchmark.

4.4 Power

4.5 Violations

4.6 Read Port Impact Estimation

Real processors have a limited number of read ports for each predictor component. If the memory dependence predictor has two ports, it can dispatch up to two memory operations per cycle. Gem5 does not model this behaviour by default, as it is a low-level detail that is difficult to represent accurately at the level of abstraction it simulates. Given the importance of this aspect to our proposed technique, we estimate the potential impact on IPC when read ports are limited. We generously assume that predictions always arrive within a single cycle for both Store Sets and PHAST. We assign a number of read ports to the memory dependence predictor for each processor model, as listed in Table 1. Dispatch is modelled to block when the number of memory operations dispatched in a cycle exceeds the number maximum number of read ports. This includes both loads and stores for Store Sets and only loads for PHAST. The count of in-use ports resets to zero each cycle. This provides a rough estimation of the impact of predicted non-dependent loads on workloads with a high density of memory operations. For clarity, **the number of read ports does not affect any other results presented in this paper**.

4.7 Profile Analysis & Threshold Sensitivity

5 Threats to Validity

- encoding space (cite cooperative memory disambig)
- intended for specific target compilation

6 Future Work

-serverless workloads
-JITs

7 Conclusion